

1. Contexto de la actividad

En esta evaluación grupal deberán resolver un problema aplicado utilizando los conceptos trabajados durante el ramo de **Fundamentos de Programación**, incorporando además principios iniciales de **Programación Orientada a Objetos (POO)**.

El desafío consiste en desarrollar un programa en **Java** que permita asignar ciudadanos a colegios públicos que funcionarán como locales de votación en la **Región Metropolitana de Santiago de Chile**.

La asignación deberá considerar la distancia geográfica entre cada ciudadano y cada colegio, usando coordenadas de **latitud** y **longitud**.

A diferencia de ejercicios anteriores, en esta actividad los datos no deberán escribirse directamente dentro del código. Los grupos deberán leer la información desde archivos **CSV**, recorrer su contenido y transformar cada fila en objetos Java.

2. Objetivo general

Desarrollar una solución funcional en Java que asigne ciudadanos a locales de votación, considerando distancia, disponibilidad de cupos y reglas de asignación, aplicando correctamente estructuras de control, arreglos o listas, métodos, clases, objetos, lectura de archivos y principios básicos de POO.

3. Objetivos específicos

Al finalizar la actividad, el grupo deberá ser capaz de:

- Modelar entidades del problema mediante clases Java.
- Utilizar atributos, constructores, métodos, getters y setters.
- Leer archivos CSV desde Java.
- Convertir filas de archivos CSV en objetos del programa.
- Aplicar estructuras condicionales y ciclos.
- Utilizar arreglos o listas para almacenar ciudadanos, colegios y asignaciones.
- Calcular distancias entre coordenadas geográficas.
- Ordenar alternativas de asignación según distancia.
- Validar reglas de negocio relacionadas con cupos y distancia máxima.
- Generar una salida clara y verificable del proceso de asignación.
- Elaborar un informe formal académico explicando la solución desarrollada.

4. Descripción del problema

Una institución necesita asignar ciudadanos a locales de votación en la Región Metropolitana.

Para esto, cuenta con dos archivos CSV:

- ciudadanos_santiago.csv
- colegios_santiago.csv

Cada ciudadano posee:

- ID.
- RUT ficticio.
- Nombre.
- Comuna.
- Latitud.
- Longitud.

Cada colegio posee:

- Código.

Nombre.
Comuna.
Latitud.
Longitud.
Capacidad máxima de ciudadanos asignados.

La lógica principal es la siguiente:

Cada ciudadano debe ser asignado preferentemente al colegio disponible más cercano.
La asignación normal solo debe realizarse si el colegio está a una distancia menor o igual a **5 km**.

Cada colegio tiene una capacidad máxima de **3 ciudadanos**.

Si el colegio más cercano ya no tiene cupos, se debe buscar el siguiente colegio más cercano disponible.

Si no existe ningún colegio disponible dentro de 5 km, el ciudadano podrá ser asignado de manera **excepcional** al colegio disponible más cercano, aunque esté a más de 5 km.

Si no existen colegios con cupos disponibles, el ciudadano debe quedar marcado como **sin asignación**.

El programa debe mostrar claramente si una asignación fue:

Normal.

Excepcional por distancia.

No asignada por falta de cupos.

5. Archivos CSV obligatorios

Los grupos deberán utilizar los archivos entregados por el docente:

ciudadanos_santiago.csv

colegios_santiago.csv

Estos archivos forman parte obligatoria del problema. No se permitirá resolver la actividad copiando manualmente todos los datos dentro del código fuente.

Los grupos pueden agregar más ciudadanos o colegios si lo desean, pero deben mantener los registros entregados originalmente.

6. Requisito extra obligatorio: lectura de archivos CSV en Java

El programa deberá leer ambos archivos CSV desde Java (investigar).

7. Fórmula de distancia requerida

Para calcular la distancia entre dos coordenadas geográficas, deberán utilizar la fórmula de Haversine.

La distancia se debe calcular en kilómetros.

8. Requisitos técnicos obligatorios

El programa deberá ser desarrollado en **Java** y cumplir como mínimo con los siguientes requisitos:

8.1. Uso obligatorio de POO

El programa debe contener, al menos, las siguientes clases:

Clase sugerida	Responsabilidad
Ciudadano	Representar a una persona que debe ser asignada a un local de votación.
Colegio	Representar un local de votación con ubicación y capacidad máxima.
Asignacion	Representar el resultado de asignar un ciudadano a un colegio.
CalculadoraDistancia	Calcular la distancia entre dos coordenadas usando Haversine.
LectorCSV	Leer los archivos CSV y construir listas de objetos.
AsignadorVotacion	Contener la lógica principal de asignación.
Main	Ejecutar el programa, cargar los datos y mostrar los resultados.

Los grupos pueden agregar más clases si lo consideran necesario.

8.2. Conceptos de fundamentos de programación que deben utilizarse

La solución debe evidenciar el uso correcto de:

- Variables.
- Tipos de datos.
- Operadores aritméticos y lógicos.
- Condicionales if, else if, else.
- Ciclos for, while o for-each.
- Métodos con parámetros y retorno.
- Arreglos o ArrayList.
- Lectura de archivos.
- Conversión de datos desde String a double o int.
- Búsqueda de elementos.

Ordenamiento por distancia.
Validación de condiciones.
Separación de responsabilidades mediante métodos.
Comentarios explicativos en partes relevantes del código.

8.3. Restricciones importantes

El programa debe ejecutarse correctamente.
No se aceptarán soluciones que solo contengan pseudocódigo.
No se aceptarán soluciones hechas en otro lenguaje distinto a Java.
No se aceptarán trabajos individuales.
No se permitirá entregar solamente capturas de pantalla del código.
El código debe ser funcional y comprobable.
El programa debe leer los datos desde los archivos CSV.
No se permite cargar manualmente todos los ciudadanos y colegios dentro del código.
Si el código no compila o no ejecuta, el trabajo será evaluado con la nota mínima.
Si el programa no lee los archivos CSV, se aplicará una penalización importante.
Si el grupo no utiliza POO de forma clara, se aplicará una penalización importante.
Si el grupo no implementa cálculo de distancia real entre latitud y longitud, se aplicará una penalización importante.
No se permite resolver la asignación manualmente escribiendo directamente el resultado final en pantalla. La asignación debe ser calculada por el programa.

9. Lógica mínima esperada

El programa debe realizar, como mínimo, el siguiente flujo:

Leer el archivo ciudadanos_santiago.csv.
Crear objetos Ciudadano a partir de las filas leídas.
Leer el archivo colegios_santiago.csv.
Crear objetos Colegio a partir de las filas leídas.
Para cada ciudadano:
Calcular la distancia hacia cada colegio.
Ordenar los colegios desde el más cercano al más lejano.
Buscar el primer colegio con cupo disponible y distancia menor o igual a 5 km.
Si no existe, buscar el colegio disponible más cercano aunque esté a más de 5 km.
Registrar si la asignación fue normal o excepcional.
Si no existe ningún colegio con cupo, marcar como no asignado.
Mostrar el resumen final.

10. Entregables

Cada grupo deberá entregar:

10.1. Informe formal en PDF

El informe debe tener estructura académica y contener:

Portada.
Integrantes del grupo.
Introducción.
Descripción del problema.
Explicación de la solución propuesta.
Diseño de clases.

Explicación de la lectura de archivos CSV.
Explicación de la fórmula de distancia utilizada.
Explicación del algoritmo de asignación.
Capturas de ejecución del programa.
Resultados obtenidos.
Conclusión.
Código fuente pegado en anexos o referencia al archivo entregado.

10.2. Código fuente Java

El código puede entregarse de una de las siguientes formas:

Archivos .java separados.
Proyecto comprimido en .zip.
Código fuente incluido como anexo en el informe, siempre que también sea posible comprobar su ejecución.
Cualquier trabajo sin código fuente funcional tendrá la nota mínima.

Se recomienda entregar el proyecto en .zip junto con el PDF.

11. Consideraciones sobre el trabajo grupal

Esta actividad es estrictamente grupal. No se recibirán trabajos individuales.
Cada integrante debe participar activamente en el análisis, diseño, programación, pruebas e informe.

En la portada del informe se debe indicar claramente:

Nombre completo de cada integrante.
Rol o aporte principal de cada integrante.
Sección del curso.
Nombre del docente.
Fecha de entrega.

Si un integrante aparece en la portada, pero no participó efectivamente, el grupo deberá informarlo al docente antes de la entrega.

12. Preguntas orientadoras para el informe

El informe debe responder, al menos, las siguientes preguntas:

¿Qué clases fueron creadas y qué responsabilidad tiene cada una?
¿Por qué se utilizó POO para resolver este problema?
¿Cómo se realizó la lectura de archivos CSV?
¿Qué dificultades surgieron al transformar texto del CSV en datos numéricos?
¿Cómo se calcula la distancia entre un ciudadano y un colegio?
¿Cómo se decide qué colegio asignar a cada ciudadano?
¿Qué ocurre cuando un colegio alcanza su capacidad máxima?
¿Qué ocurre cuando no existe un colegio disponible dentro de 5 km?
¿Qué casos de prueba realizaron?
¿Qué dificultades tuvo el grupo durante el desarrollo?
¿Qué mejoras podrían implementarse en una versión futura?